UART sender 시뮬레이션 — 시나리오와 결과 한눈에

sender = 값 24 비트 + 형식 sel → 글자 줄 ("12:34:56.78" CR LF) 을 TX FIFO 에 한 글자씩 넣는 블록

검증	시나리오	확인한 것	결과	장
글자 MUX	① 스톱워치 시간 한 줄	시간 24 비트 → 13 글자	**13 / 13**	3
(조합회로)	② 거리 0 · 7 · 58 · 400	세 자리 0 채움 + cm	**28 / 28**	4
tb_sd1_char	③ 온습도 3 줄 + sel 5 · 6 · 7	45%25C · 잘못된 sel → ERROR	**45 / 45**	5
sender FSM	(1) 시간 한 줄	go → busy 13 클럭 · 1 클럭 1 글자	**err 0**	6
tb_sd2_sender	(2) ~ (4) 거리 · 온습도 · ERROR	busy = 글자 수 (7 · 8 · 7)	**err 0**	7
	(5) TX FIFO full 20 클럭	멈췄다 이어서 · 손실 0	**busy 33**	8
	(6) busy 중 send_go	무시 (ERROR 안 섞임)	**err 0**	9
	(7) 도중 데이터 변경	go 순간 값 유지	**err 0**	10
	(8) 5 종 연달아	busy 0 다음 클럭 go 받음 · 54 글자	**54 / 54**	11 · 12
	(9) (10) go 전 · LF 직전 full	첫 · 마지막 글자도 기다림	**busy 18 · 17**	13
	(11) 마지막 클럭 send_go	버려짐 → busy 0 보고 go	**err 0**	14
	(12) 도중 rst	즉시 IDLE · 다음 줄 정상	**cnt 5**	15
	(13) 랜덤 200 줄 + 무작위 full	기대값 모델과 모든 글자 비교	**err 0**	16
Top (sender	시간 한 줄 전체	busy 130 ns ↔ 선 13.49 ms	**13 / 13**	17
+ TX FIFO	시작 확대 60 ~ 265 ns	go → FIFO → uart_tx → start 비트	**35 ns**	18
+ uart_tx)	한 글자 프레임 "1"	LSB 먼저 · 비트 104.2 µs	**rcnt 1**	19
tb_sd3_top	5 종 연달아 54 글자	56.25 ms 쉬지 않고 전송	**67 / 67**	20
	5 종 busy 확대	TX FIFO 64 칸 꽉 참 0 번	**full 0**	21

세 TB 모두 TOTAL MISMATCH = 0 (Vivado 2022.2 xsim, clk 100 MHz = 10 ns, UART 9600 bps)

파형 읽는 법 — i_send_data 24 비트는 "숫자 여러 개를 비트로 붙인 값"

파형에는 16 진수 (예 528f28) 로 보임 → 칸을 나눠 10 진수로 읽으면 시간 · 거리 · 온습도

sel 0 · 1 · 2 (스톱워치 · 시계 · 타이머)



16 진	2 진 (시 | 분 | 초 | 1/100 초)	읽은 값
528f28	0101 0 | 010 100 | 0 1111 0 | 010 1000	→ **10 : 20 : 30 . 40**
645c4e	0110 0 | 100 010 | 1 1100 0 | 100 1110	→ **12 : 34 : 56 . 78**
bf7de3	1011 1 | 111 011 | 1 1101 1 | 110 0011	→ **23 : 59 : 59 . 99**

보내는 줄
시간 13 글자
12:34:56.78 CR LF
거리 7 글자
400cm CR LF
온습도 8 글자
45%25C CR LF
ERROR 7 글자
sel 5 · 6 · 7

sel 3 거리



000190 → 190_{16} = **400 cm**

sel 4 온습도

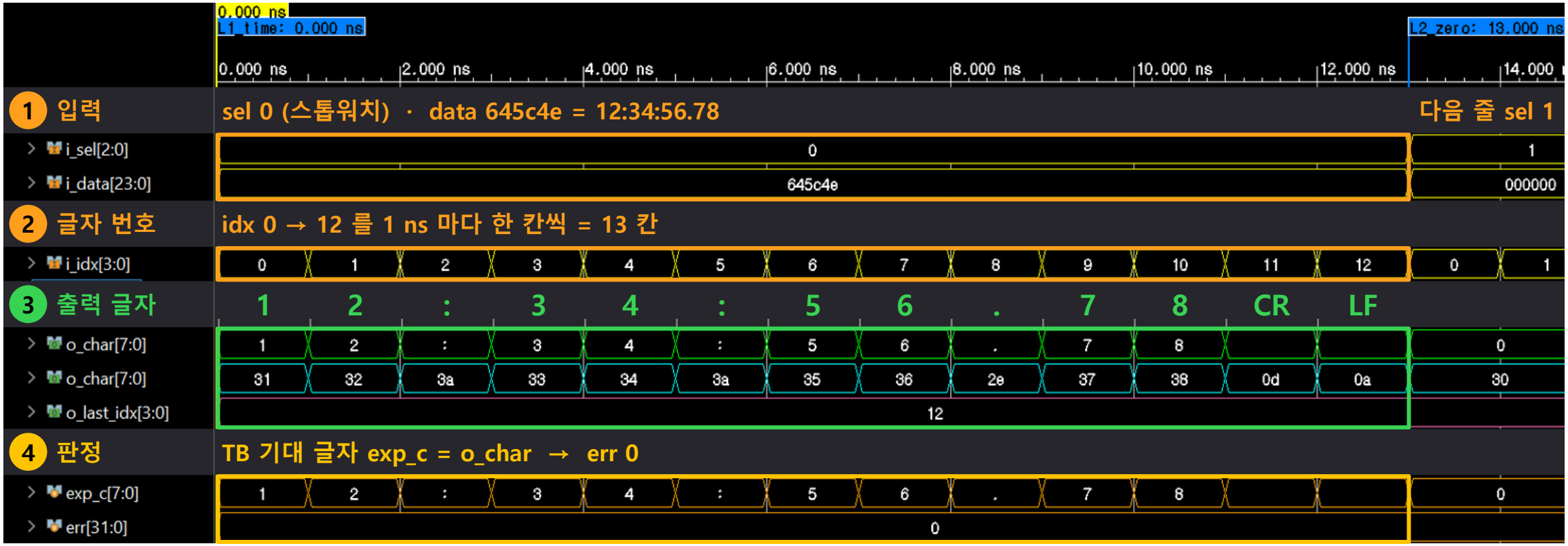


002d19 → 2d = 45 % · 19 = 25 ℃ → **45%25C**

글자 = 8'h30 + 자리 숫자 (예 "1" = 31, ":" = 3a, CR = 0d, LF = 0a) — 파형의 o_tx_push_data 줄

글자 MUX — 시간 24 비트 645c4e 를 글자 13 개 "12:34:56.78" + CR LF 로 바꿈

시나리오 sel 0 + data 645c4e 고정, idx 0 → 12 를 1 ns 마다 바꿔 각 칸의 글자 확인 (tb_sd1_char)



1 입력 : 시간 24 비트

645c4e = 시 5 · 분 6 · 초 6 · 1/100 초 7 비트
= 12 : 34 : 56 . 78

2 글자 번호 idx

sender FSM 이 줄 c_idx 자리.
0 → 12 = 글자 13 개

3 출력 o_char

숫자 = 8'h30 + 자리값
31 32 3a 33 34 … 0d 0a
마지막 칸 o_last_idx = 12

4 판정

TB 가 따로 계산한 exp_c 와
칸마다 비교 → err 0

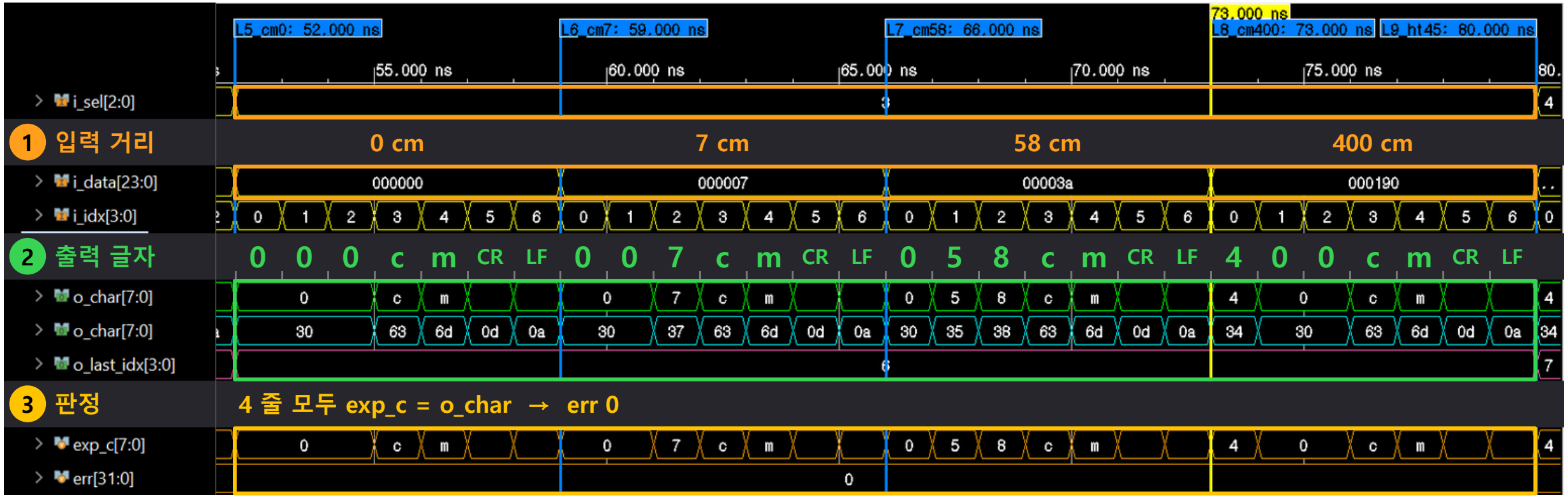
결과

13 / 13 글자 일치
err 0
조합회로 : idx 가 바뀌면 같은 시각에 글자도 바뀜

색 입력 출력 → TX FIFO 판정

글자 MUX — 거리는 늘 세 자리 + "cm" : 0 → 000cm · 7 → 007cm · 58 → 058cm · 400 → 400cm

시나리오 sel 3 (거리), data 0 · 7 · 58 · 400 (최소 · 한 자리 · 두 자리 · 최대) 한 줄씩, 줄마다 idx 0 → 6



1 입력 : 거리 9 비트

data = {15'b0, cm 9 비트}
000000 · 000007 · 00003a · 000190

2 출력 7 글자

백 · 십 · 일 자리 + c m CR LF
빈 자리는 0 → 늘 3 자리
o_last_idx = 6

3 판정

4 줄 × 7 글자를 exp_c 와 비교
→ err 0

4 파형 읽는 법

같은 값 (30 30 30) 이 이어지면
Vivado 는 칸 하나로 그림
→ 위 큰 글자가 실제 글자

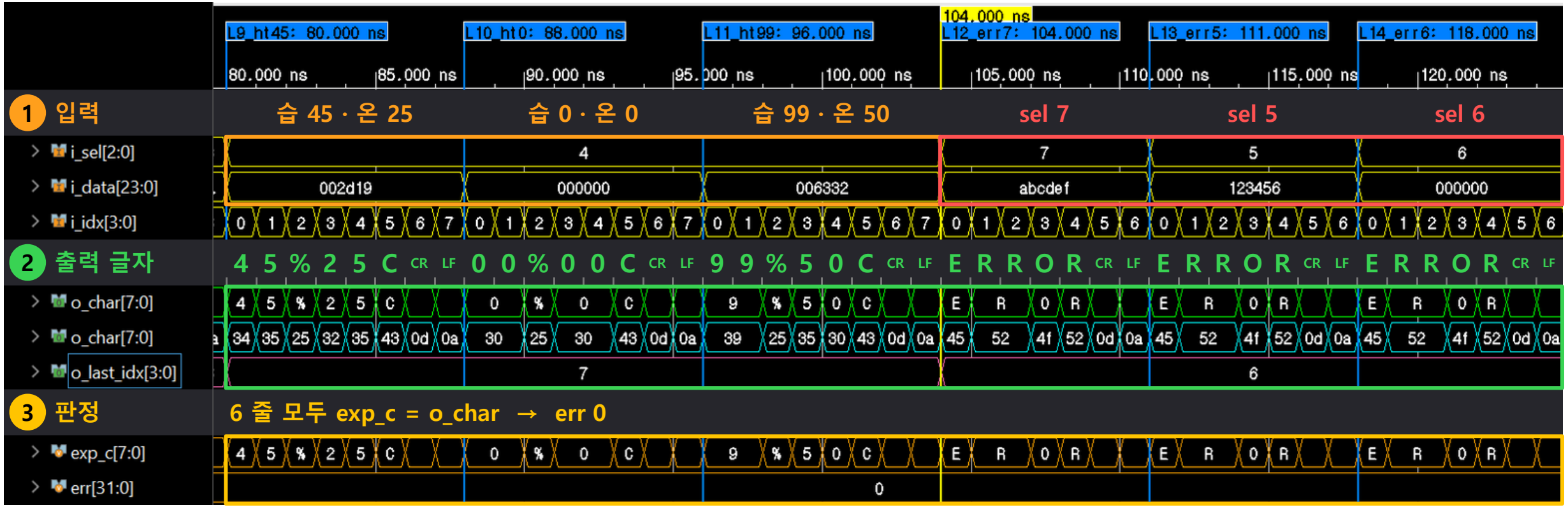
결과

28 / 28 글자 일치
err 0
경계값 0 · 400 포함

색 입력 출력 → TX FIFO 판정

글자 MUX — 온습도는 "45%25C", 안 쓰는 sel 5 · 6 · 7 은 데이터와 상관없이 "ERROR"

시나리오 sel 4 로 습 · 온 (45 · 25 / 0 · 0 / 99 · 50), 이어서 sel 7 · 5 · 6 에 아무 데이터 (abcdef · 123456 · 000000)



1 입력 : 습 · 온

data = {8'b0, 습 8, 온 8}
002d19 → 2d = 45 · 19 = 25

2 출력

온습도 8 글자 (o_last_idx 7)
ERROR 7 글자 (o_last_idx 6)

3 sel 5 · 6 · 7

data 가 abcdef · 123456 이어도
늘 E R R O R → 잘못된 sel 방어

4 판정

6 줄 45 글자 exp_c 비교
→ err 0

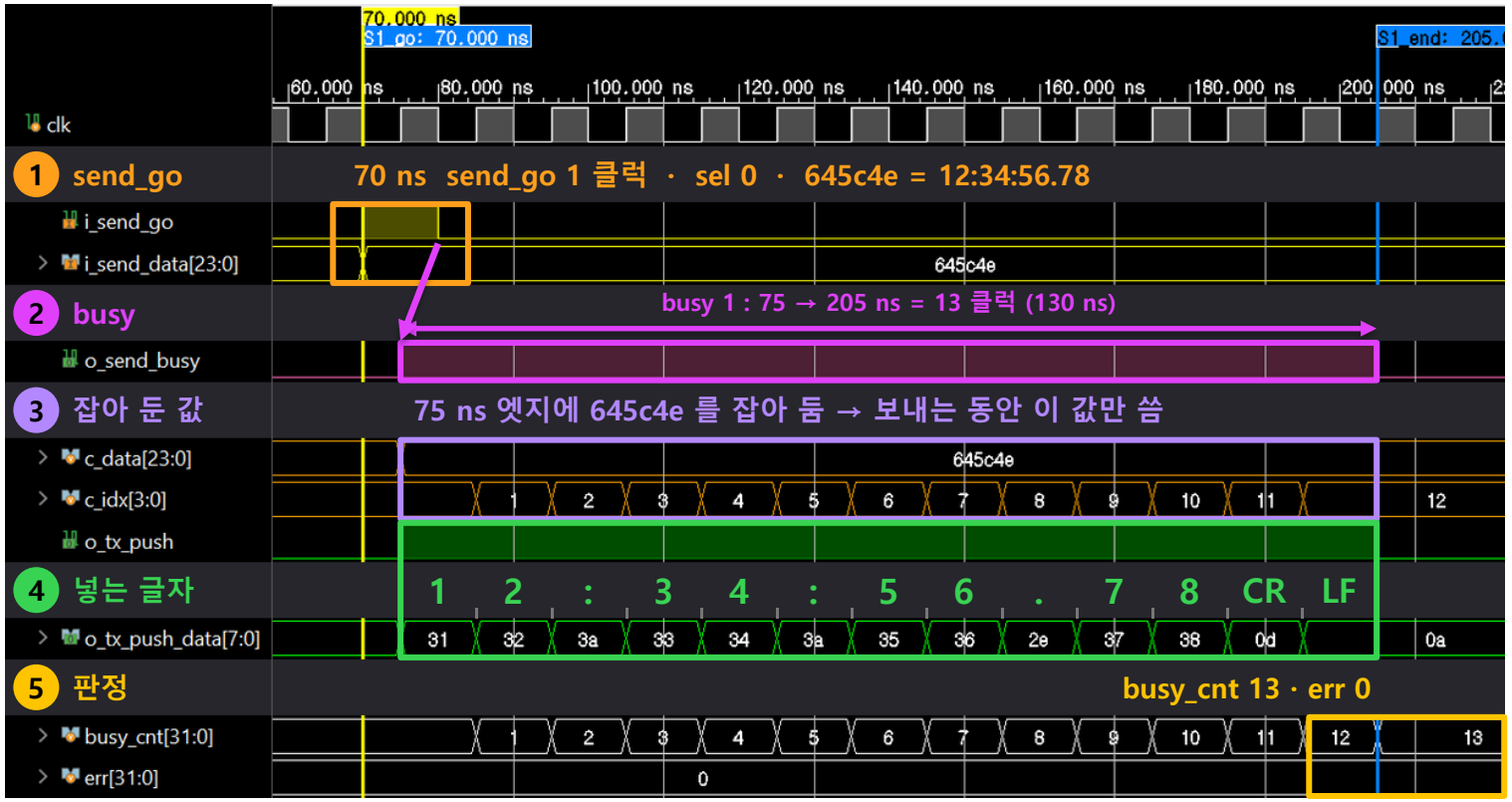
결과

45 / 45 글자 일치
err 0
최소 0 · 최대 99 · 잘못된 sel 3 개 포함

색 입력 출력 → TX FIFO 판정 무시 · 리셋

sender — send_go 한 번에 글자 13 개를 1 클럭에 1 개씩 TX FIFO 에 넣음 (busy 13 클럭)

시나리오 (1) TX FIFO 비어 있음 (full 0), 70 ns 하강 엣지에 sel 0 + 12:34:56.78 로 send_go 1 클럭



1 send_go 70 ns
하강 엣지에 1 클럭 펄스
sel 0 · data 645c4e

2 busy 13 클럭
다음 상승 엣지 75 ns 에 SEND
13 글자 = 13 클럭 뒤 IDLE

3 c_data · c_idx
75 ns 에 645c4e 저장
c_idx 0 → 12
클럭마다 +1

4 push · 글자
push 1 이 13 클럭 계속
31 32 3a … 0d 0a

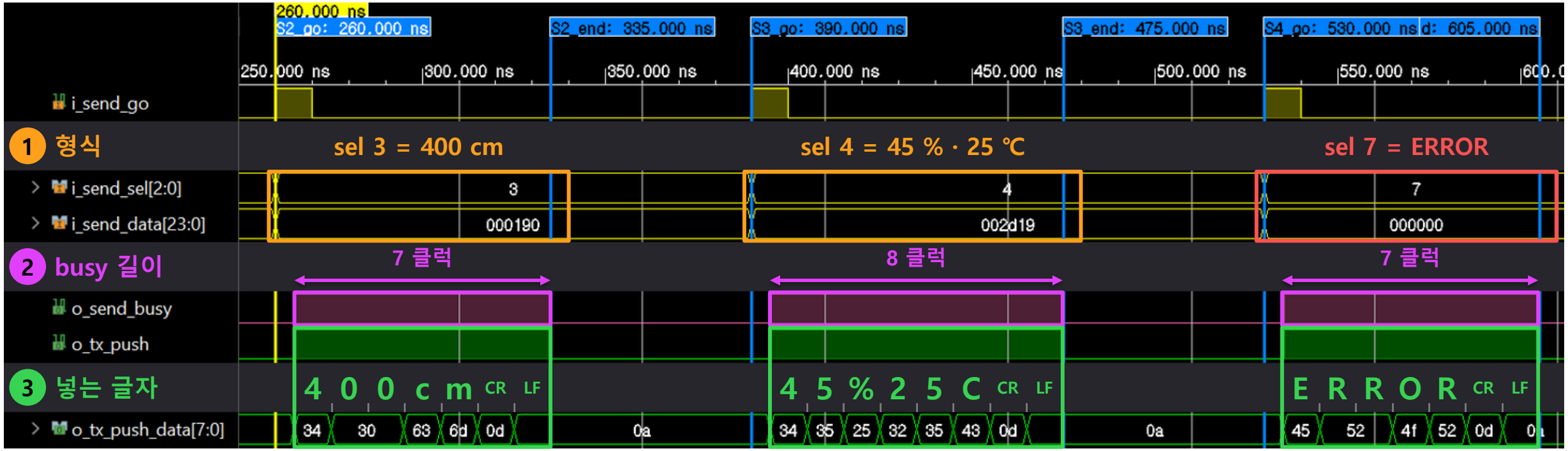
5 판정
busy_cnt 13
받아 적은 13 글자 = 기대
err 0

결과
busy 13 클럭 = 130 ns
글자 13 / 13 일치 · err 0
go → SEND 반 클럭 (5 ns)

색 입력 제어 (busy) 잡아 둔 값 출력 → TX FIFO 판정

sender — 응답 길이만큼 busy : 거리 7 · 온습도 8 · ERROR 7 클럭

시나리오 (2) ~ (4) 거리 400 cm · 온습도 45 % 25 ℃ · ERROR 를 한 줄씩 send_go (줄 사이 쉼)



1 형식 3 종

sel 3 거리 · sel 4 온습도
sel 7 ERROR (data 무시)

2 busy 길이 = 글자 수

o_last_idx 로 줄 끝을 앎
7 · 8 · 7 클럭

3 넣는 글자

400cm · 45%25C · ERROR
+ CR LF

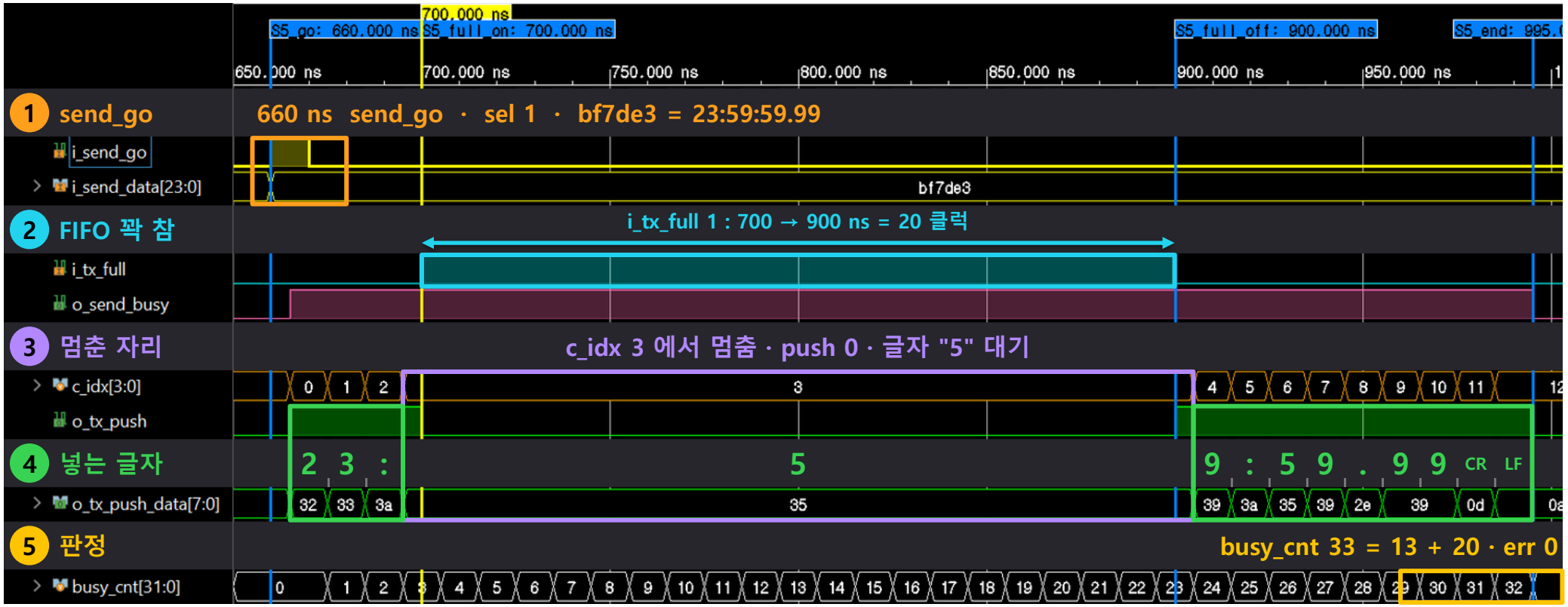
결과

busy 7 · 8 · 7 클럭
글자 22 / 22 일치 · err 0
고정 길이 13 이 아니라
형식마다 다름

색 입력 제어 (busy) 출력 → TX FIFO 무시 · 리셋

sender — TX FIFO 가 꽉 차면 그 글자에서 멈췄다가, 풀리면 이어서 넣음 (글자 손실 0)

시나리오 (5) 23:59:59.99 를 보내다가 3 글자 ("23:") 넣은 뒤 TB 가 i_tx_full 을 20 클럭 동안 1



1 send_go 660 ns

sel 1 · bf7de3 = 23:59:59.99

2 full 20 클럭

700 → 900 ns
TX FIFO 가 꽉 찬 흉내

3 멈춤

push = SEND & ~full → 0
c_idx 3 · 글자 "5" 그대로

4 이어서

900 ns 뒤 5 9 : 5 9 . 9 9 CR LF
순서 그대로

5 판정

busy_cnt 33 = 13 + 20
full 중 push 0 번 · err 0

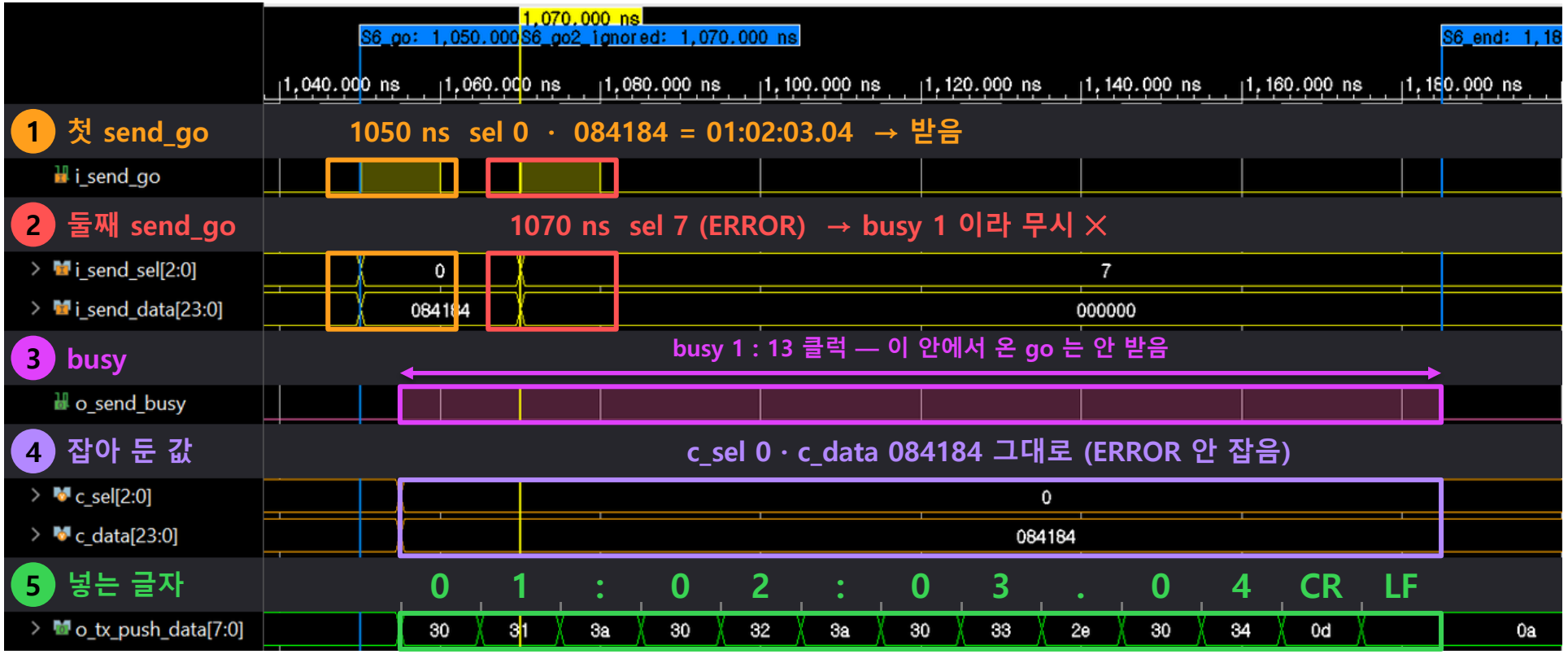
결과

busy 33 클럭 = 글자 13 + 기다림 20
글자 13 / 13 순서 그대로
full 중 push 0 (흘림 없음)

색 입력 FIFO 꽉 참 잡아 둔 값 출력 → TX FIFO 판정

sender — 보내는 중 (busy 1) 에 온 send_go 는 무시 : "ERROR" 가 섞이지 않음

시나리오 (6) 1050 ns 에 01:02:03.04 를 보내기 시작, 2 클럭 뒤 1070 ns 에 sel 7 (ERROR) send_go 를 또 줌



1 첫 go

1050 ns · sel 0 · 084184
→ 1055 ns 에 SEND

2 둘째 go

1070 ns · sel 7
SEND 중이라 FSM 이 안 봄

3 busy

1055 → 1185 ns 13 클럭
그대로 끝남

4 잡아 둔 값

c_sel 0 · c_data 084184
바뀌지 않음

5 글자

01:02:03.04 CR LF
"E" (45) 없음

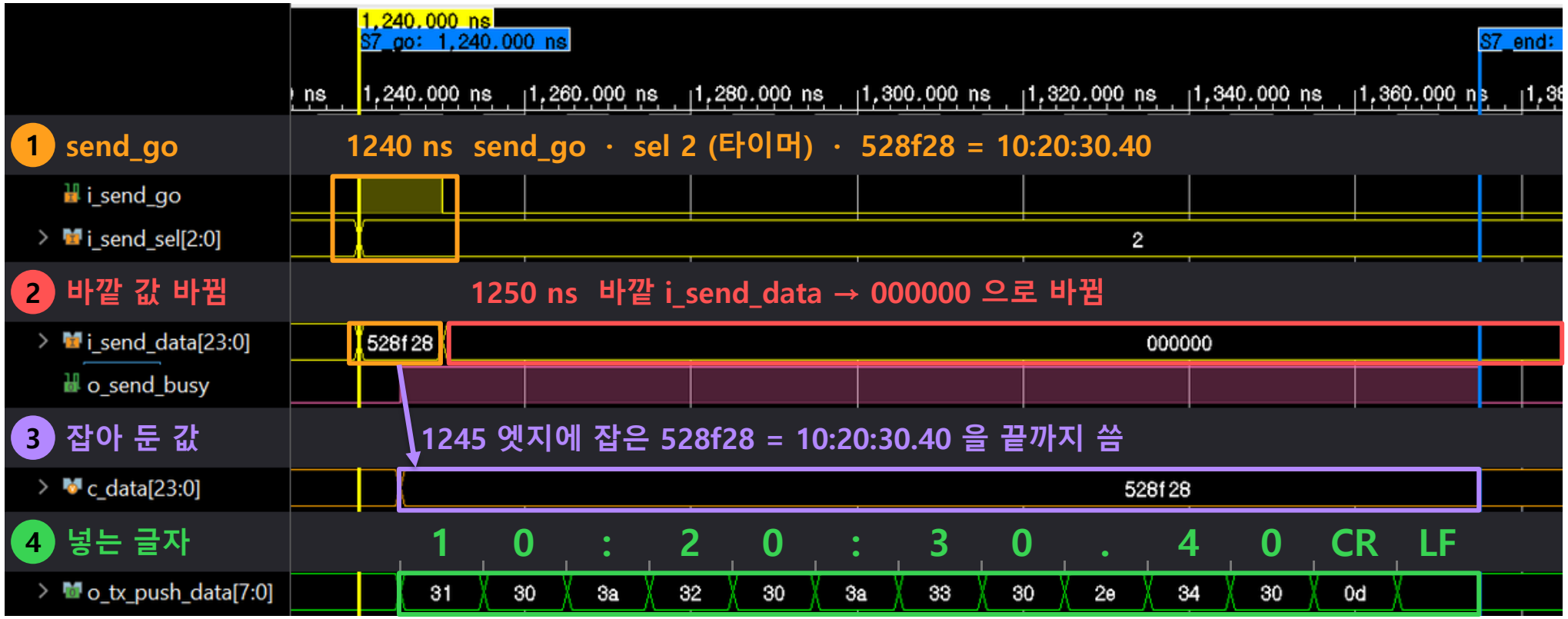
결과

둘째 go 무시
글자 13 / 13 = 01:02:03.04
→ U_CONTROL 은 busy 0 일 때만 go

색 입력 무시 · 리셋 제어 (busy) 잡아 둔 값 출력 → TX FIFO

sender — 보내는 도중 바깥 값이 바뀌어도 send_go 순간 값 (528f28) 을 끝까지 보냄

시나리오 (7) 1240 ns 에 10:20:30.40 으로 send_go, 1 클럭 뒤 TB 가 i_send_data 를 000000 으로 바꿈



1 send_go 순간 값

528f28 = 시 10 · 분 20 · 초 30 · 1/100 초 40

2 바깥 값 변경

1250 ns 에 000000 (보내는 중 시계 바뀜)

3 c_data

1245 ns 엣지에 저장 줄 끝까지 528f28

4 글자

10:20:30.40 CR LF 13 글자 그대로

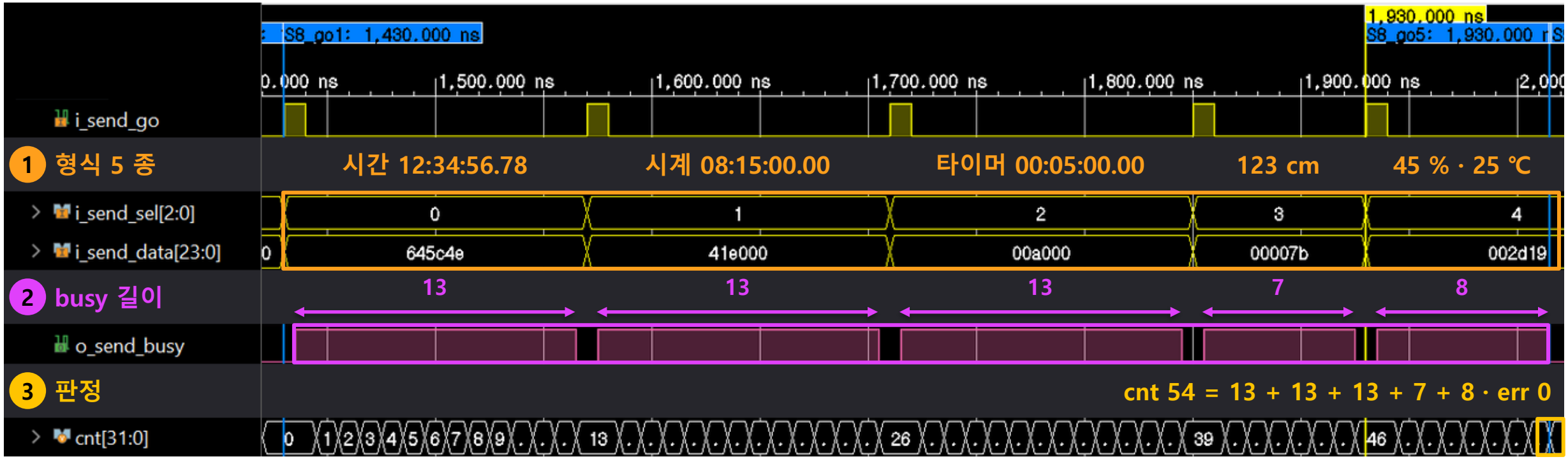
결과

c_data 유지
글자 13 / 13 = 10:20:30.40
한 줄 안 값 안 섞임

색 입력 무시 · 리셋 잡아 둔 값 출력 → TX FIFO

sender — 5 종을 연달아 : busy 가 0 이 된 다음 클럭에 바로 다음 send_go → 54 글자

시나리오 (8) /auto all 흉내 — sel 0 · 1 · 2 · 3 · 4 를 busy 가 0 되자마자 차례로 send_go



1 형식 5 종

스톱워치 · 시계 · 타이머
· 거리 · 온습도

2 busy 5 구간

13 · 13 · 13 · 7 · 8 클럭
사이 빈 클럭 1 개씩

3 판정

받아 적은 54 글자 = 기대
err 0

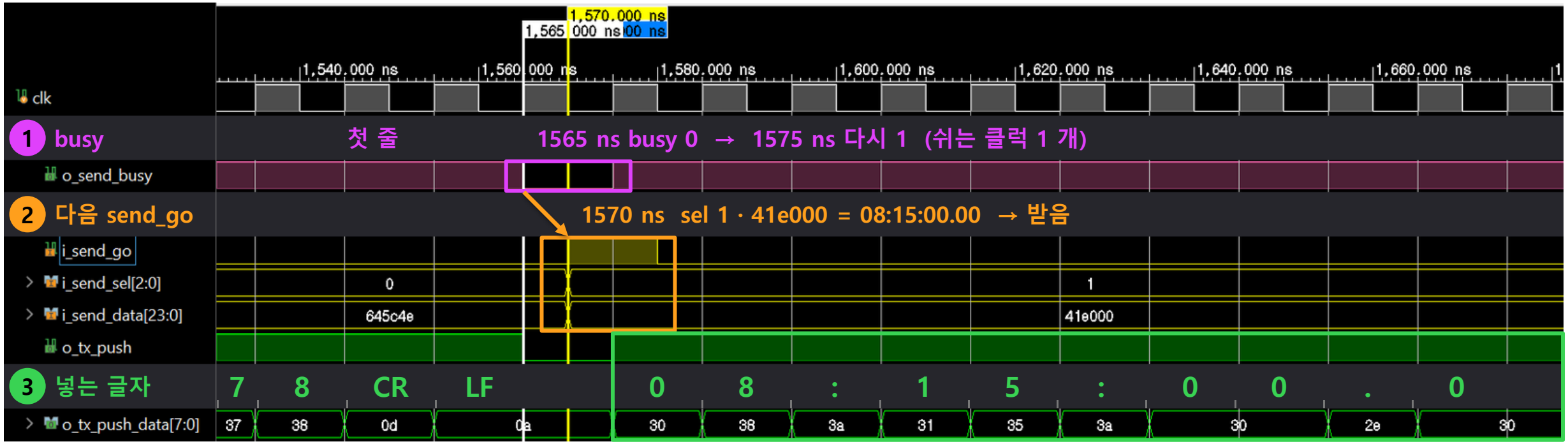
결과

54 / 54 글자 일치
1430 → 2015 ns (58.5 클럭)
TX FIFO 64 칸 안

색 입력 제어 (busy) 판정

sender — busy 0 을 본 바로 다음 클럭의 send_go 는 받아짐 (빈 틈 1 클럭)

시나리오 (8) 확대 — 첫 줄 (스톱워치) 이 1565 ns 에 끝나고 1570 ns 하강 엣지에 시계 줄 send_go



1 1565 ns busy 0

첫 줄 LF 를 넣은 엣지에서 IDLE

2 1570 ns send_go

U_CONTROL 이 busy 0 을 보고
다음 하강 엣지에 go

3 1575 ns 다음 줄 시작

0 8 : 1 5 : 0 0 . 0 0 ...
쉬는 클럭 1 개

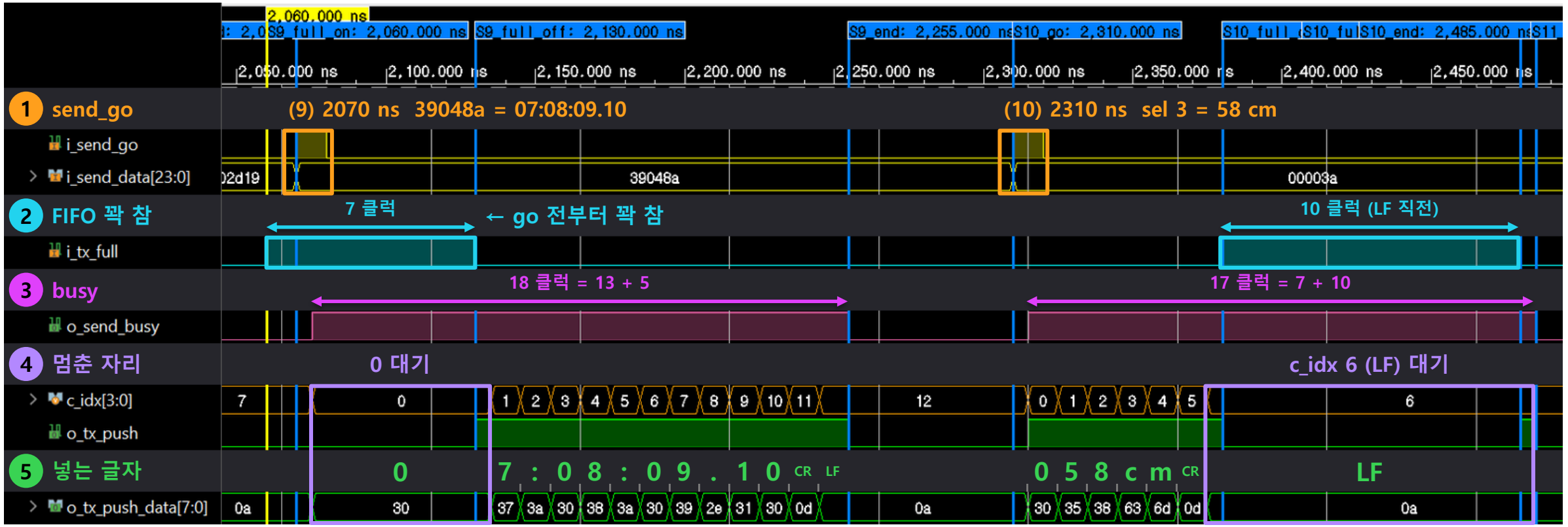
결과

busy 0 다음 클럭 go = 받음
줄 사이 빈 틈 10 ns
↔ 다음 장 : busy 1 마지막 클럭 go = 버림

색 입력 제어 (busy) 출력 → TX FIFO

sender — full 이 go 전부터 와도, 마지막 LF 직전에 와도 기다렸다가 글자 하나 안 빠짐

시나리오 (9) full 을 먼저 켠 채 send_go (5 클럭 뒤 풀림) (10) "058cm CR" 까지 넣은 뒤 LF 앞에서 full 10 클럭



1 send_go 두 번
(9) 07:08:09.10
(10) 58 cm

2 full
(9) 2060 → 2130 ns
(10) 2380 → 2480 ns

3 busy 늘어남
(9) 13 + 5 = 18 클럭
(10) 7 + 10 = 17 클럭

4 멈춘 자리
(9) 첫 글자 c_idx 0
(10) 마지막 LF c_idx 6

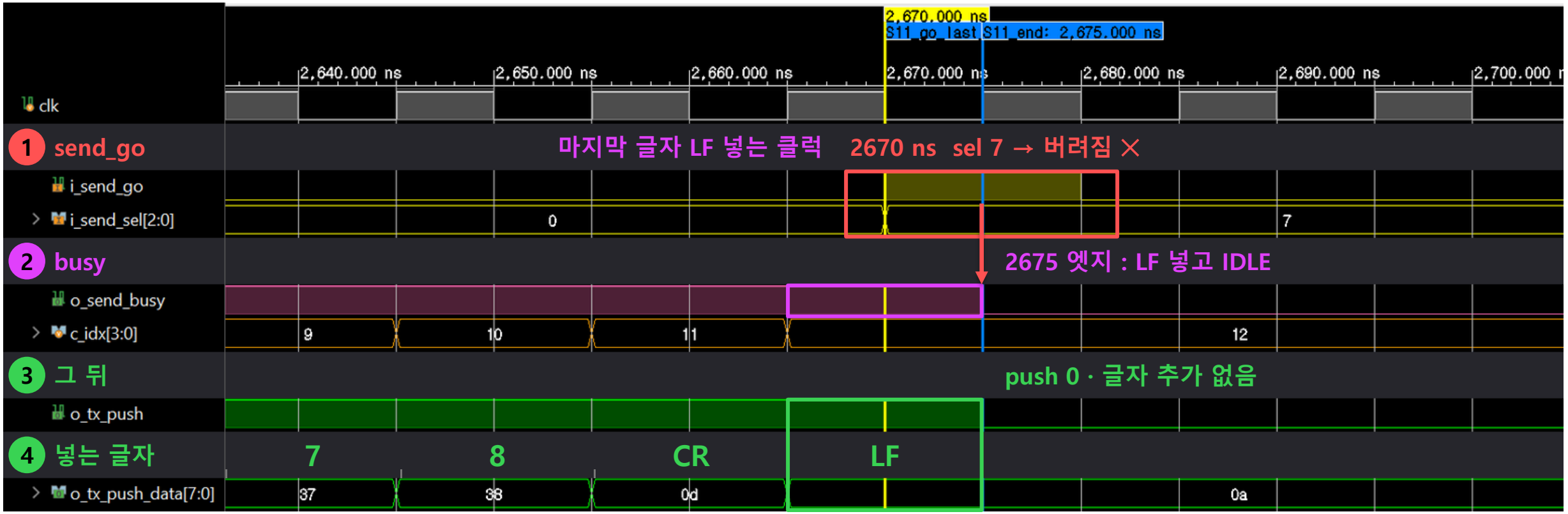
5 글자
07:08:09.10 CR LF
058cm CR LF 순서 그대로

결과
두 줄 20 / 20 글자 일치
busy = 글자 수 + full 클럭
첫 · 마지막 글자도 안 빠짐

색 입력 FIFO 꽉 참 제어 (busy) 잡아 둔 값 출력 → TX FIFO

sender — busy 1 인 마지막 클럭 (LF 넣는 엣지) 에 걸친 send_go 는 버려짐

시나리오 (11) 한 줄의 마지막 글자 LF 를 넣는 2675 ns 엣지에 sel 7 send_go (2670 ~ 2680 ns) 를 겹쳐 줌



1 2670 ns send_go

아직 busy 1 (SEND)
FSM 은 SEND 에서 go 를 안 봄

2 2675 ns 엣지

LF 를 넣고 IDLE
같은 엣지라 go 를 못 받음

3 그 뒤

push 0 · 글자 0 개
(TB cnt 13 그대로)

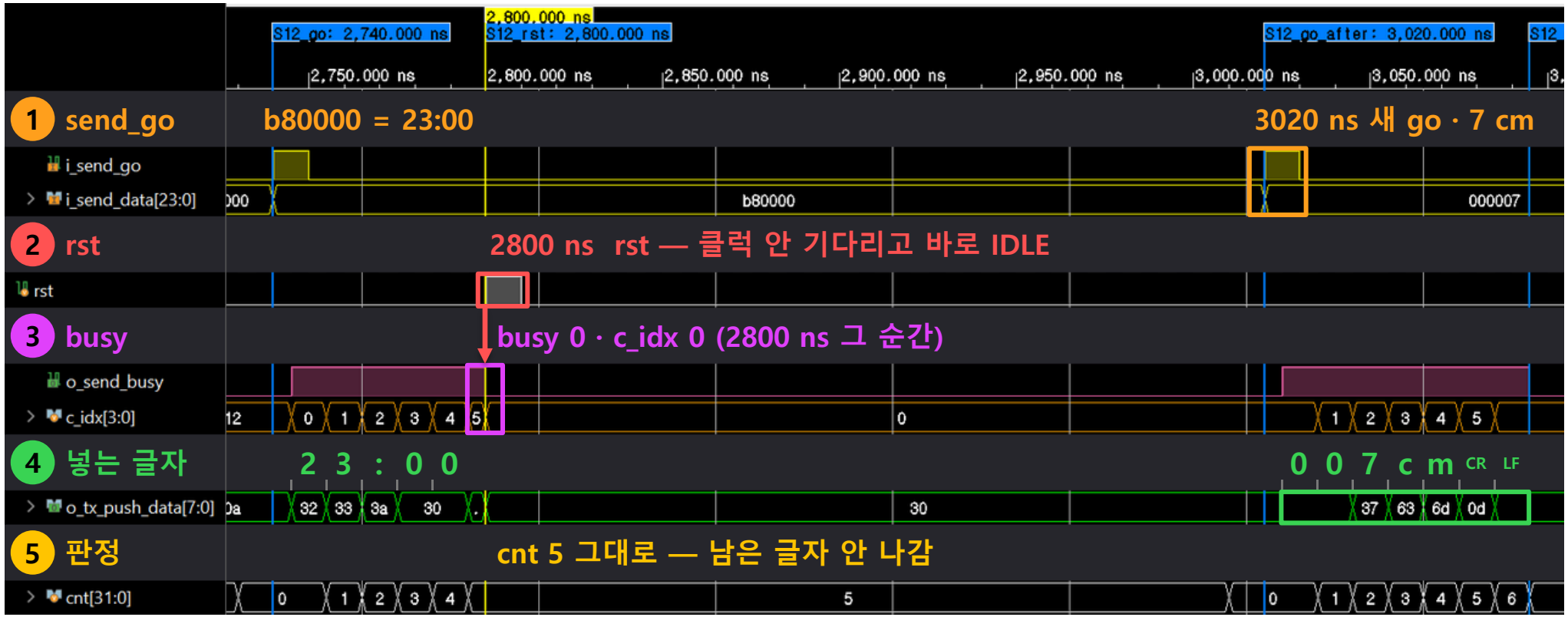
결과

마지막 클럭 go = 버림
규칙 : busy 0 을 본 뒤 go
(앞 장 : 그러면 다음 클럭에 받음)

색 무시 · 리셋 제어 (busy) 출력 → TX FIFO

sender — 보내는 도중 rst : 그 순간 IDLE · busy 0, 남은 글자는 안 나가고 다음 go 는 정상

시나리오 (12) 23:00:00.00 을 보내다 5 글자 ("23:00") 넣은 뒤 2800 ns 에 rst 1 클럭, 이어서 3020 ns 에 7 cm



1 send_go 2740 ns

b80000 = 23:00:00.00
5 글자 넣는 중

2 rst 2800 ns

비동기 리셋
클럭 엣지 안 기다림

3 IDLE

busy 0 · c_idx 0
2800 ns 바로

4 다음 go

3020 ns 7 cm
→ 007cm CR LF
정상

5 판정

cnt 5 그대로
err 0

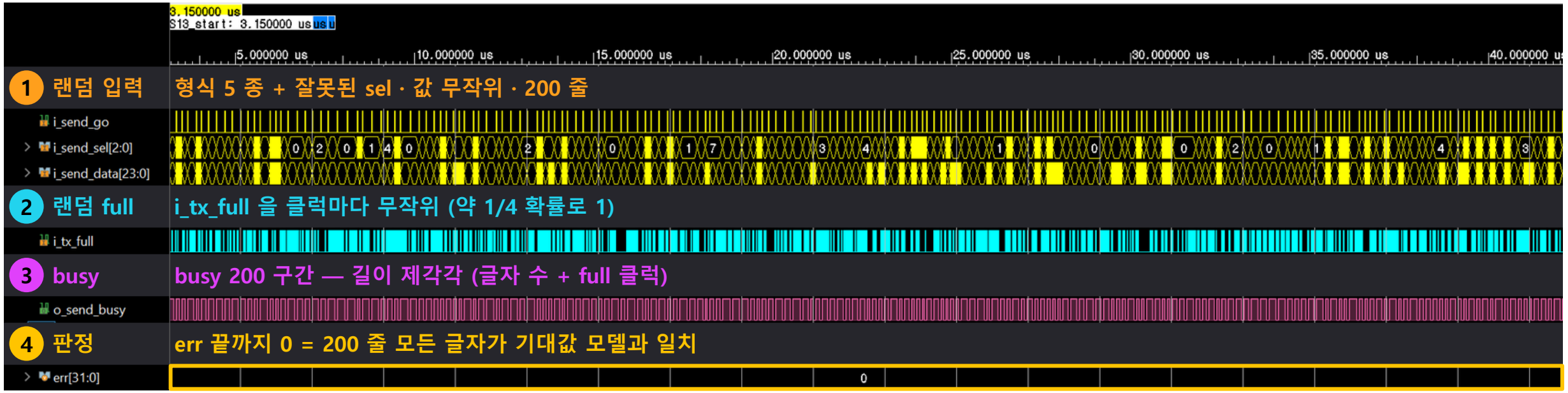
결과

rst 즉시 IDLE
남은 8 글자 안 나감
rst 뒤 다음 줄 정상

색 무시 · 리셋 입력 제어 (busy) 출력 → TX FIFO 판정

sender — 랜덤 200 줄 + 무작위 full : 모든 글자 일치 (err 0)

시나리오 (13) 3.15 µs 부터 형식 · 값 · full 위치를 무작위로 200 줄, 기대 글자는 TB 의 make_exp 모델이 따로 계산



1 랜덤 입력

sel 0 ~ 7, 시 · 분 · 초 · cm · 습 · 온 범위 안 무작위

2 랜덤 full

클럭마다 약 1/4
어느 글자에서든 멈춤

3 busy

200 구간
길이 = 글자 + 기다림

4 판정

받아 적은 글자 vs
make_exp → err 0

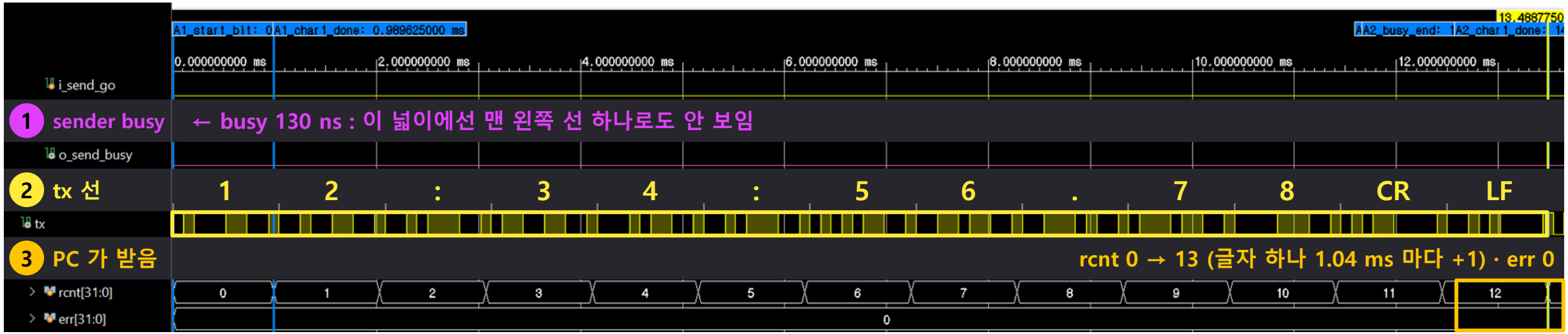
결과

200 줄 err 0
TOTAL MISMATCH 0
(1) ~ (12) 정해진 시나리오
+ (13) 랜덤

색 입력 FIFO 꽉 참 제어 (busy) 판정

Top — sender 는 130 ns 만에 끝, 선으로 나가는 13.5 ms 는 TX FIFO · uart_tx 몫

시나리오 tb_sd3_top : sender + TX FIFO 64 + uart_tx 연결, 70 ns 에 12:34:56.78 send_go, TB 가 PC 처럼 tx 선을 읽음



1 sender busy 130 ns

13 글자를 TX FIFO 에 넣으면 끝
(75 → 205 ns)

2 TX FIFO

13 글자를 받아 두고 uart_tx 가 하나씩 꺼냄

3 tx 선 13.49 ms

9600 bps · 글자 하나 10 비트
= 1.04 ms × 13

4 PC 흉내 TB

stop 비트 가운데서 받아 적음
rcnt 13 · err 0

결과

busy 130 ns ↔ 선 13.49 ms
약 10 만 배 차이
13 / 13 글자 일치

색 제어 (busy) tx 선 · PC 판정

Top — 첫 글자가 sender → TX FIFO → uart_tx 로 넘어가 105 ns 에 start 비트가 나감

시나리오 tb_sd3_top 시작 확대 : 70 ns send_go (sel 0 · 12:34:56.78) 직후 60 ~ 265 ns



1 75 ns ~ sender

1 클럭에 1 글자 push
205 ns 까지 13 글자

2 85 ns FIFO

첫 글자 저장
wptr 1 · empty 0

3 95 ns 꺼냄

uart_tx 가 pop
rptr 1

4 95 ns uart_tx

c_state START
data_reg = "1" (31)

5 105 ns tx

start 비트 0
선으로 나가기 시작

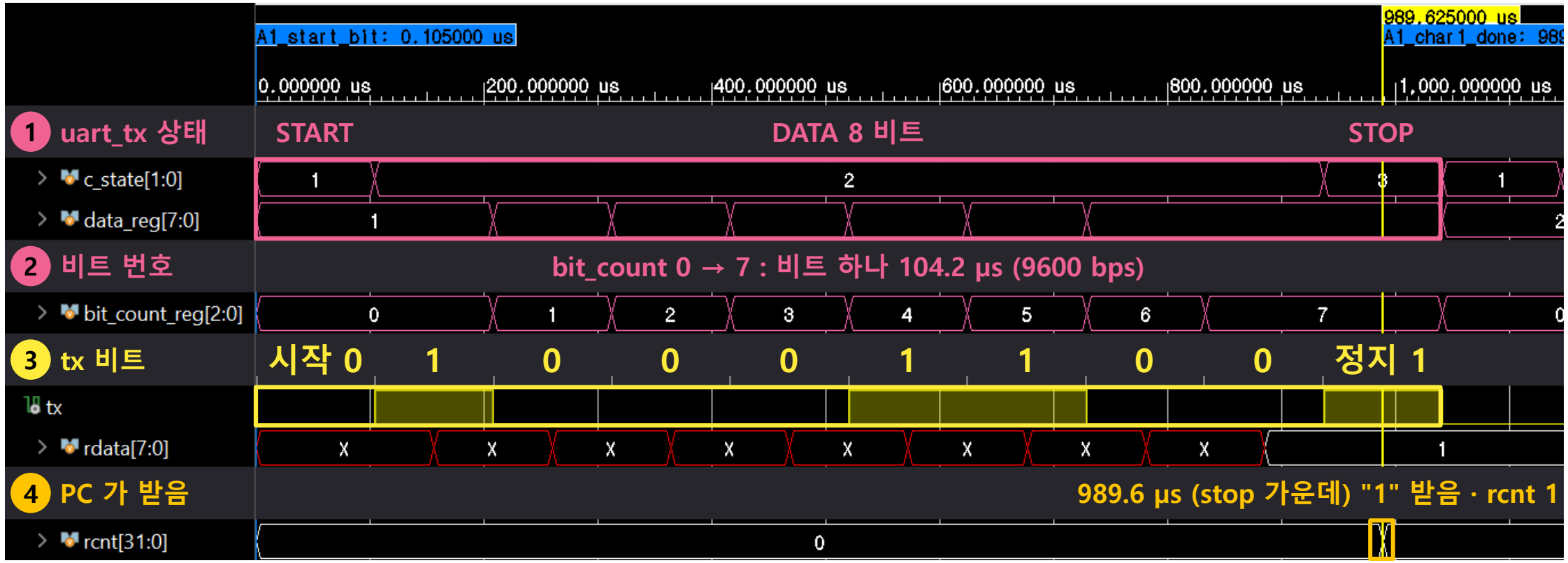
결과

go → start 비트 35 ns
205 ns : FIFO 에 12 글자 대기
(wptr 13 · rptr 1)

색 출력 → TX FIFO · TX FIFO · uart_tx · tx 선 · PC

uart_tx — 글자 "1" (31 = 0011_0001) 을 LSB 먼저 1 0 0 0 1 1 0 0, 비트 하나 104.2 µs

시나리오 tb_sd3_top 첫 글자 프레임 확대 : 105 ns start 비트부터 stop 비트까지 (0 ~ 1.15 ms)



1 c_state

1 START → 2 DATA → 3 STOP
→ 다음 글자 START

2 data_reg · bit_count

data_reg "1" (31)
bit_count 0 → 7

3 tx 비트열

31 = 0011_0001
LSB 먼저 → 1 0 0 0 1 1 0 0

4 PC 흉내 TB

stop 비트 가운데 989.6 µs
rdata "1" · rcnt 1

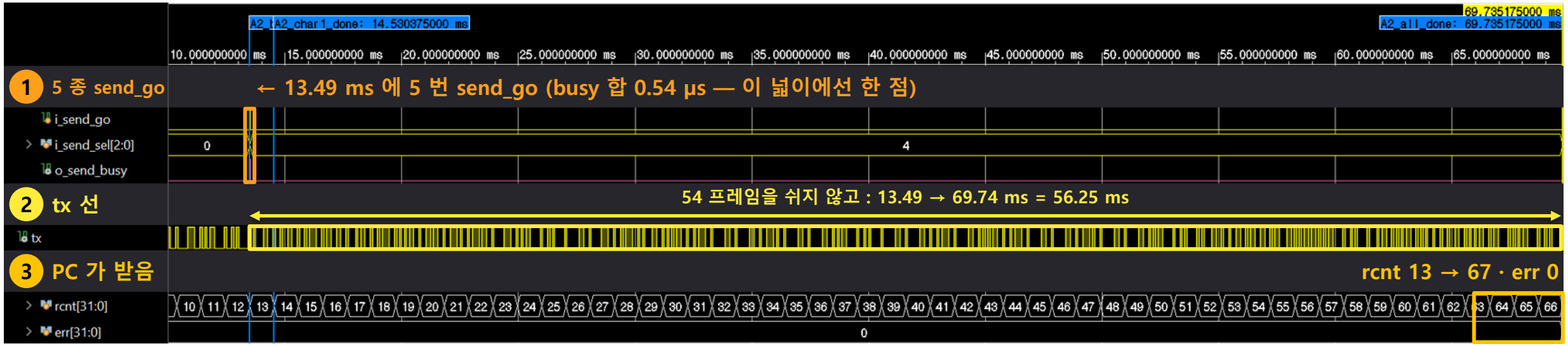
결과

한 글자 = 10 비트 = 1.04 ms
9600 bps (104.17 µs / 비트)
rdata "1" · err 0

색 uart_tx tx 선 · PC 판정

Top — 5 종 54 글자를 한꺼번에 넣어도 TX FIFO 가 받아 두고 uart_tx 가 56 ms 동안 쉬지 않고 보냄

시나리오 tb_sd3_top 둘째 부분 : 첫 줄이 끝난 13.49 ms 에 /auto all 흉내로 sel 0 ~ 4 send_go 5 번



1 5 번 go

13,488,780 → 13,489,280 ns
busy 합 54 클럭

2 TX FIFO

54 글자를 0.6 µs 만에 받음
(64 칸 안)

3 tx 54 프레임

1.04 ms × 54 = 56.25 ms
프레임 사이 쉼 없음

4 PC 흉내 TB

받은 글자 67 개
= 첫 줄 13 + 54 · err 0

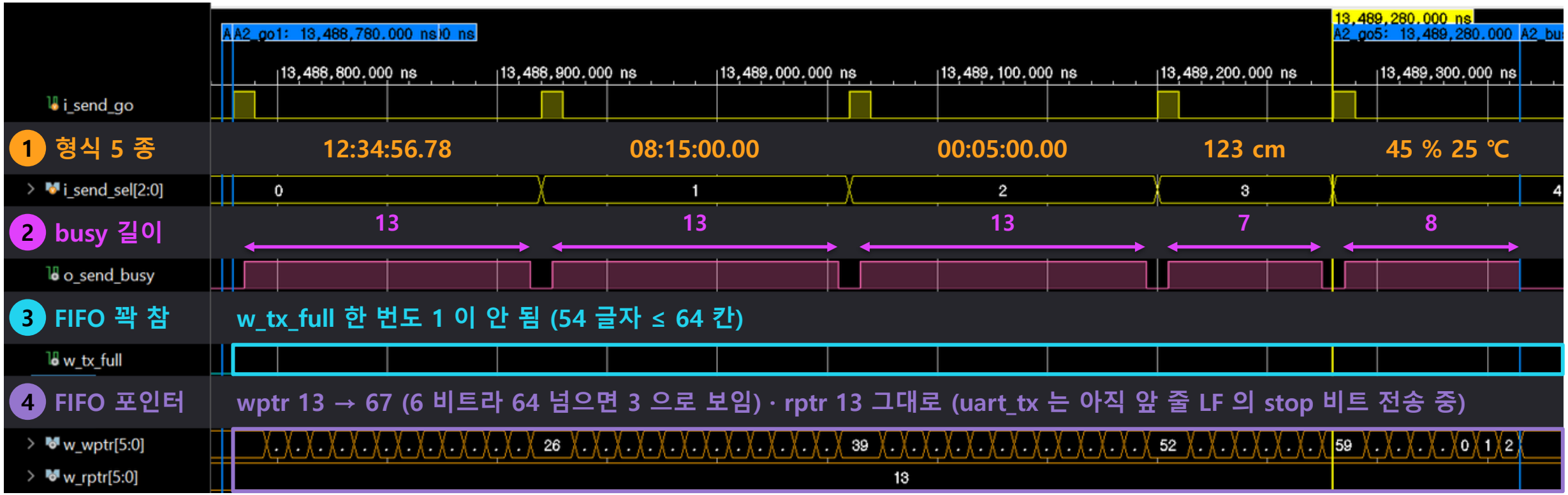
결과

67 / 67 글자 일치
sender 0.6 µs ↔ 선 56 ms
FIFO 꽉 참 0 번 (다음 장)

색 입력 tx 선 · PC 판정

Top — 5 줄 54 글자를 0.6 µs 만에 넣어도 TX FIFO (64 칸) 는 한 번도 꽉 차지 않음

시나리오 앞 장의 13.49 ms 부분 확대 : 13,488,770 ~ 13,489,385 ns



1 5 번 go

busy 0 다음 클럭마다 다음 줄 send_go

2 busy

13 · 13 · 13 · 7 · 8 클럭 = 54 글자

3 full 0

TX FIFO 64 칸 > 54 글자 → sender 가 안 기다림

4 포인터

wptr 13 → 67 (= 3)
rptr 13 (앞 줄 stop 비트 중)

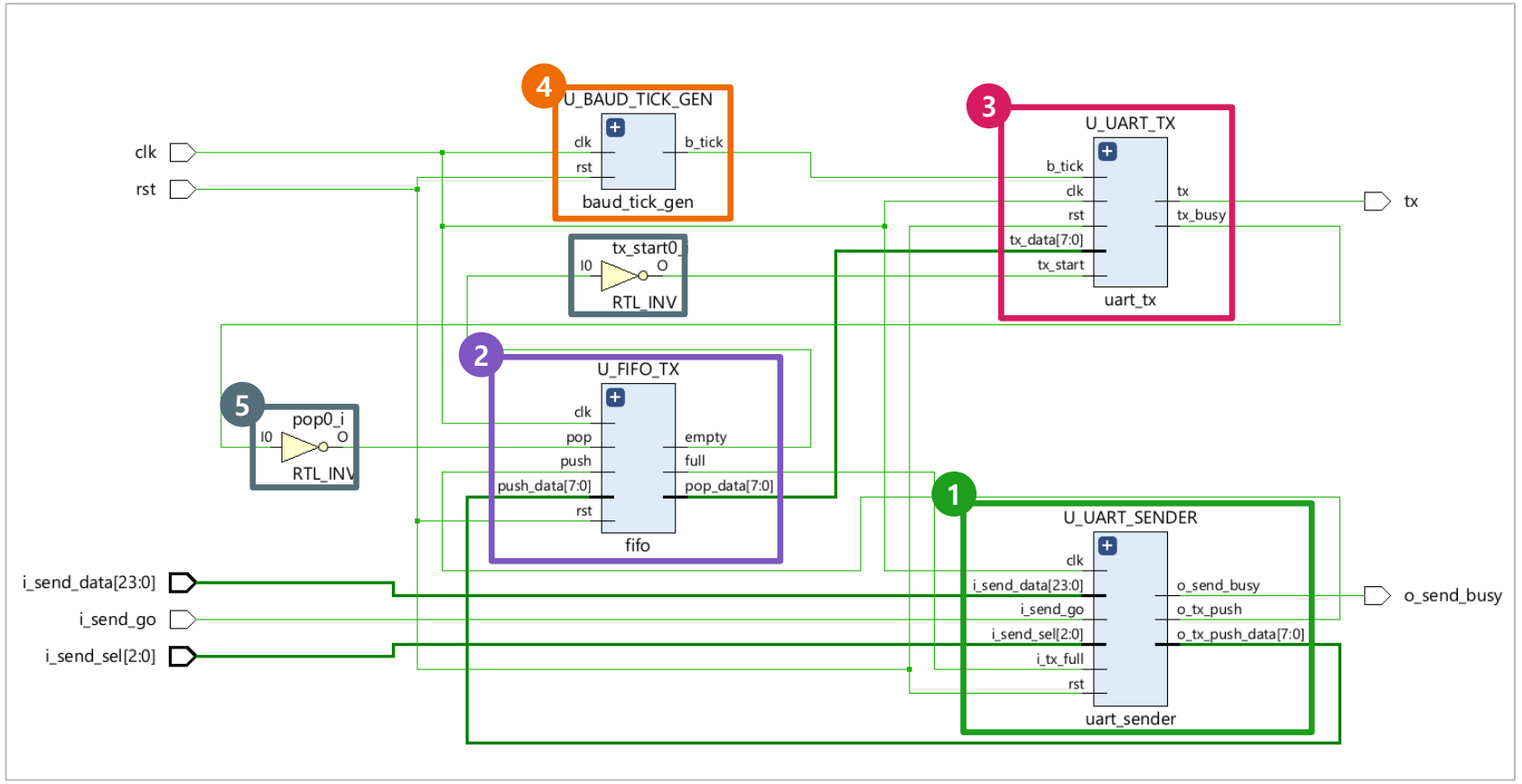
결과

full 0 번 · 손실 0
54 글자 ≤ 64 칸
→ busy 가 짧게 끝남

색 입력 제어 (busy) FIFO 꽉 참 TX FIFO

Top RTL — sender → TX FIFO (64) → uart_tx → tx, 사이 연결은 NOT 두 개

시나리오 sender_tx_top : 내 uart_sender 에 숙제 fifo · uart_tx · baud_tick_gen 을 이어 붙인 시험용 Top (RTL Schematic)



1 U_UART_SENDER (내 설계)

send_go · sel · data → 1 클럭 1 글자
o_tx_push · o_tx_push_data
i_tx_full ← FIFO full

2 U_FIFO_TX (64 칸)

#(.DEPTH(64)) 로 숙제 fifo 크기만 바꿈
push ← sender · pop_data → uart_tx

3 U_UART_TX

tx_start 이면 tx_data 를
start · 8 비트 · stop 으로 tx 에

4 U_BAUD_TICK_GEN

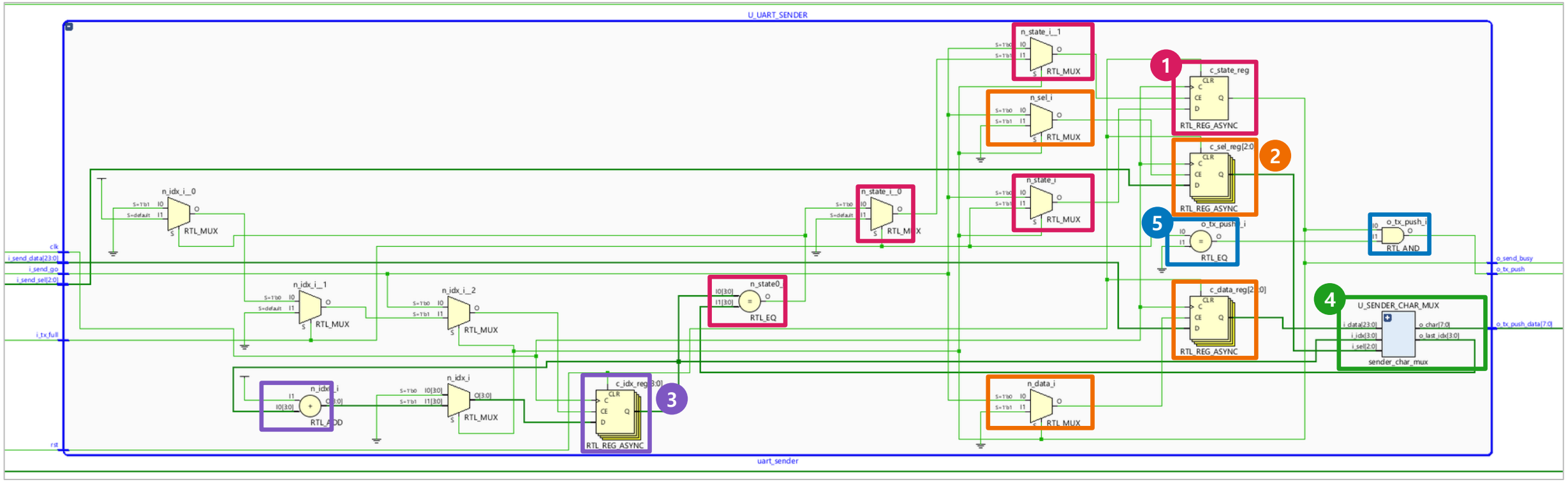
9600 × 16 tick (b_tick)
→ uart_tx 비트 박자

5 RTL_INV 2 개 (연결)

tx_start = ~empty (글자 있으면 시작)
pop = ~tx_busy (안 바쁘면 꺼냄)

uart_sender RTL — 상태 1 비트 (c_state) + 잡아 두는 레지스터 3 개 + 글자 MUX

시나리오 U_UART_SENDER 펼친 화면 : 레지스터는 모두 RTL_REG_ASYNC (비동기 rst), 래치 없음



1 CU : c_state (IDLE · SEND)

c_state_reg 1 비트
n_state MUX + RTL_EQ
(c_idx == 마지막 칸 → IDLE)

2 DP : c_sel · c_data

go 때만 새 값 (n_sel · n_data MUX)
보내는 동안 바깥 값 무시

3 DP : c_idx 카운터

RTL_ADD (+1)
4 비트 레지스터

4 U_SENDER_CHAR_MUX

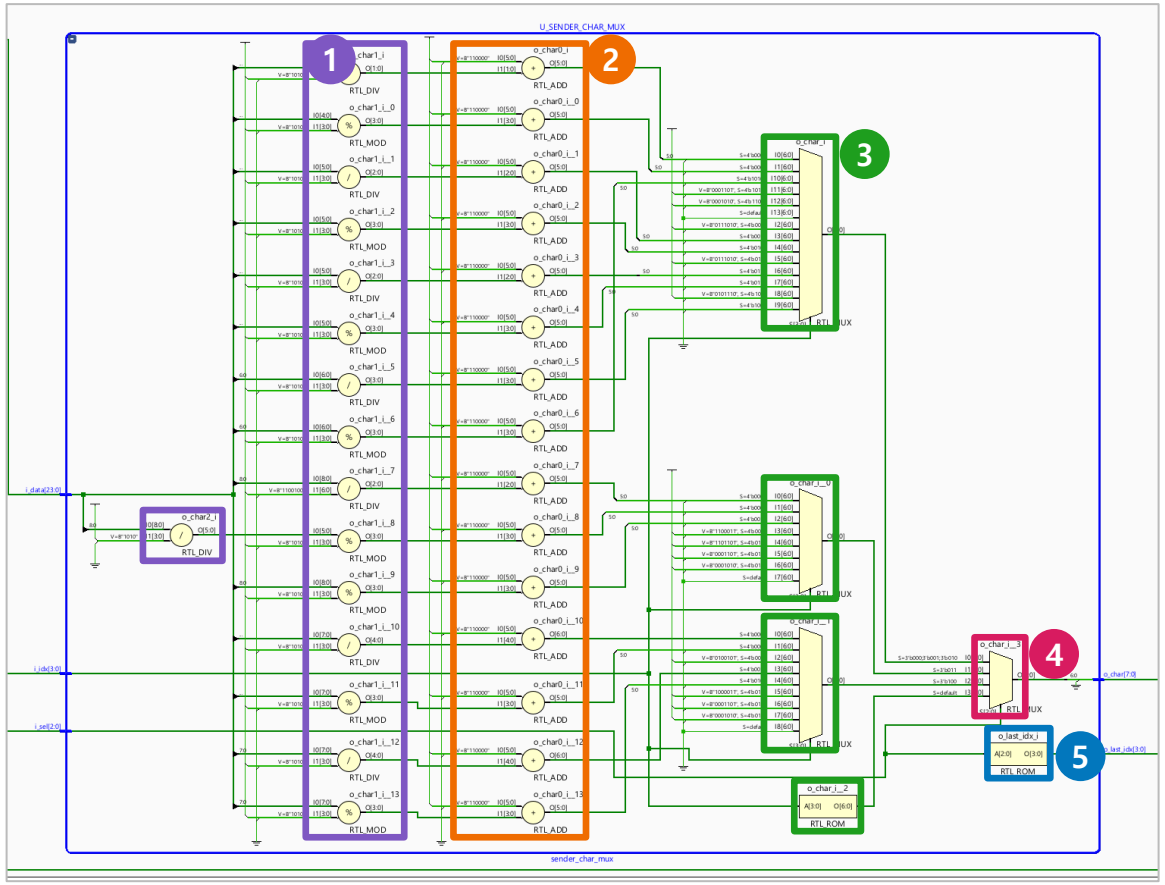
sel · data · idx → o_char · o_last_idx

5 o_tx_push

RTL_EQ (SEND) & ~i_tx_full
= RTL_AND

글자 MUX RTL — ÷10 · %10 로 자리 나누고 +"0" 해서 형식별 MUX → sel 로 고름

시나리오 U_SENDER_CHAR_MUX 펼친 화면 : 조합회로만 (레지스터 0 개)



1 자리 나누기

RTL_DIV · RTL_MOD (÷10 · %10)
시 · 분 · 초 · 1/100 · 습 · 온
거리 9 비트는 ÷100 · ÷10 · %10

2 글자로

RTL_ADD : + 8'b110000 (= "0")
숫자 0 ~ 9 → 30 ~ 39

3 형식별 MUX (idx)

시간 13 · 거리 7 · 온습도 8 칸
ERROR 는 RTL_ROM

4 sel MUX

sel 로 형식 하나 고름
→ o_char

5 o_last_idx ROM

sel → 12 · 6 · 7 · 6
(줄 끝 칸)